TABLE DES MATIÈRES

1.	Résumé Exécutif	3
2.	Ce Qui Fonctionne (■)	4
3.	Findings P0 — Critiques / Sécurité	5
4.	Findings P1 — Importants	8
5.	Findings P2 — Améliorations	10
6.	Findings P3 — Finitions UX	11
7.	Audit Supabase / RLS — Table par Table	12
8.	Architecture Membre et Permissions	13
9.	Connexions ERP	14
10.	Audit Design / UX	15
11.	Roadmap Complète	16
12.	Risques et Recommandations	17
13.	Conclusion	18

1. Résumé Exécutif

Cet audit couvre l'intégralité du module Équipe de DJAMA ERP — le frontend (/client/equipe/ et /membre/*), les routes API, les migrations Supabase, les politiques RLS, le système d'authentification des membres, et les connexions avec les autres modules. L'analyse a été conduite sans modification du code.

5 Findings P0	4 Findings P1	5 Findings P2	4 Findings P3
Critiques/Sécurité	Fonctionnalité essentielle	Améliorations	Finitions UX

Constat Central

Le module Équipe souffre d'une **fracture architecturale fondamentale** : deux systèmes parallèles coexistent sans pont fonctionnel. Le frontend chef (/client/equipe/) écrit dans les tables `team_*` (modèle solo, user_id = uid du chef, sans organization_id). L'espace membre (/membre/*) lit dans les tables `org_*` et `organization_members` (modèle multi-tenant). Ces deux architectures ne se parlent pas, rendant l'espace membre fonctionnellement cassé pour les tâches, le chat, et la synchronisation des données.

En parallèle, les permissions définies dans `organization_permissions` ne sont appliquées qu'au niveau de l'interface (sidebar masquée) mais pas au niveau des routes API — une faille de sécurité permettant à tout membre d'appeler directement n'importe quelle API ERP.

- 3 pages de l'espace membre sont fonctionnellement cassées (/membre/taches, /membre/chat, /membre/messages partiellement)
- Le chat chef et le chat membre utilisent deux tables différentes et ne se synchronisent pas
- Aucune contrainte unique n'empêche les membres dupliqués (confirmé visuellement : Nofane AMDJAD x2)
- La table employes (paie) est entièrement silotée de team_members — deux registres distincts
- L'espace membre affiche des KPIs à 0 permanents à cause du cloisonnement des données

2. Ce Qui Fonctionne ■

Malgré les problèmes structurels, plusieurs composants sont bien conçus et fonctionnels :

■ Authentification membre	Le flux /membre/login est correctement implémenté : vérification post-login dans organization_members, filtrage des membres suspendus (suspended_at IS NOT NULL), sélecteur multi-org, stockage de active_org_id dans user_metadata.
■ RLS espace membre (tables org_*)	Les tables org_messages, org_message_groups, org_shared_documents utilisent is_org_member() SECURITY DEFINER — isolation inter-organisations robuste.
■ Système de suspension	La colonne suspended_at et les fonctions is_org_member()/is_org_admin() excluent correctement les membres suspendus (migration 068). Le bug initial (admins suspendus non bloqués) a été corrigé.
■ Messagerie org (schema)	Les tables org_message_groups, org_message_group_members, org_message_reads sont well-designed : DMs, canaux, soft delete, édition, pièces jointes, receipts de lecture.
■ Documents partagés	org_shared_documents avec can_view/can_download/can_edit par destinataire, can_view_shared_doc() helper, expiration — architecture solide.
■ Layout membre dynamique	src/app/membre/layout.tsx charge organization_permissions pour construire la navigation dynamiquement. Vérifie la suspension. Appelle update_member_last_seen(). Mobile drawer responsive.
■ Dashboard membre	src/app/membre/dashboard/page.tsx lit correctement organization_members, org_messages, org_shared_documents avec scoping par org — fonctionnel.
■ Création de compte membre	L'API /api/equipe/create-member-account crée un compte Supabase Auth réel (service_role), lie auth_user_id à team_members, rate-limit 5/h. Flow correct dans son principe.
■ Rate limiting global	Les routes IA (Planning AI, Notes AI, etc.) ont toutes un rate limit. La pattern checkRateLimit est cohérente dans le codebase.
■ Modals équipe	Les formulaires de création/édition/suppression de membres, tâches, congés, réunions et timesheets fonctionnent correctement côté UX.

3. Findings P0 — Critiques / Sécurité

Les findings P0 doivent être traités en priorité absolue avant tout déploiement. Ils représentent soit des failles de sécurité, soit des fonctionnalités entièrement cassées.

P0 **#1 — Isolation multi-tenant absente sur les tables team_***

Les tables **team_members, team_tasks, team_messages, team_leaves, team_meetings** n'ont aucune colonne organization_id. Leur politique RLS est *user_id = auth.uid()* — ce qui signifie que toutes les données d'une équipe appartiennent à un seul chef (son auth.uid). Il est impossible de partager correctement ces données entre plusieurs membres d'une organisation. Toute la fonctionnalité multi-tenant du module Équipe repose sur des tables qui ignorent complètement le concept d'organisation.

Scénario : *Un chef crée une équipe → change d'organisation → les team_members de l'ancienne org sont toujours visibles. Deux organisations ne peuvent jamais partager la même infrastructure team_*.*

`supabase/migrations/equipe_schema.sql — team_members, team_tasks, team_messages, team_leaves`
`src/app/client/equipe/page.tsx — toutes les requêtes`

P0 **#2 — Espace membre /taches entièrement cassé (RLS mismatch)**

La page **/membre/taches** exécute : `.from('team_tasks').eq('user_id', teamId)` où teamId = user_metadata.team_id (l'ID du chef). Mais la politique RLS de team_tasks est *user_id = auth.uid()* — ce qui vérifie l'uid du membre connecté, pas celui du chef. Résultat : la requête retourne systématiquement 0 lignes pour chaque membre. L'espace membre est rendu inutilisable pour la gestion des tâches.

Scénario : *Membre connecté (auth.uid = 'abc') → requête WHERE user_id = 'chef-xyz' → RLS bloque (chef-xyz ≠ abc) → 0 tâches affichées.*

`src/app/membre/taches/page.tsx — ligne ~40 : .eq('user_id', teamId)`
`supabase/migrations/equipe_schema.sql — RLS team_tasks`

P0 **#3 — Chat membre et chat chef déconnectés (deux tables distinctes)**

Le chef écrit ses messages dans **team_messages** (via /client/equipe/, filtre user_id=chef.uid). L'espace membre lit aussi **team_messages** via /membre/chat, filtré par user_id = metadata.team_id. Mais la RLS de team_messages vérifie *auth.uid() = user_id* — ce qui est l'uid du membre, pas celui du chef. Résultat : les membres voient 0 message. De plus, la vraie messagerie org (org_messages) utilisée par /membre/messages est une table entièrement différente — le chef n'y écrit jamais depuis /client/equipe/.

Scénario : *Chef envoie 'Réunion vendredi' dans #général → message dans team_messages → membre ouvre son chat → RLS bloque → 0 messages affichés. Le canal de communication équipe est complètement rompu.*

`src/app/membre/chat/page.tsx — ligne 63 : .from('team_messages').eq('user_id', tid)`
`src/app/client/equipe/page.tsx — sendMessage() → team_messages`

P0 #4 — organization_permissions non appliqué côté API (contournement UI possible)

Les permissions granulaires (can_view, can_create, can_edit, can_delete, can_export) définies dans **organization_permissions** sont uniquement utilisées dans /membre/layout.tsx pour afficher ou masquer les entrées de la sidebar. Les routes API elles-mêmes (`/api/factures`, `/api/crm`, `/api/documents`, etc.) ne vérifient jamais ces permissions. Un membre avec *can_view: false* pour les Factures peut envoyer une requête HTTP directement à l'API et obtenir les données.

Scénario : *Membre avec can_view=false sur 'factures' → POST /api/factures avec son token Supabase → l'API vérifie seulement auth.getUser() (pas les permissions org) → données retournées sans filtre.*

```
src/app/membre/layout.tsx — seul endroit où organization_permissions est lu
Toutes les routes /api/* — aucune ne vérifie organization_permissions
```

P0 #5 — Membres dupliqués possibles (absence de contrainte UNIQUE)

La table **team_members** n'a pas de contrainte UNIQUE sur (user_id, email) ou (user_id, name). Il est possible d'insérer le même membre plusieurs fois — confirmé dans le screenshot fourni qui montre deux cartes 'NOFANE AMDJAD'. Aucune validation côté backend ou frontend n'empêche ce doublon. Un doublon peut causer des confusions de permissions, des tâches dupliquées, et des erreurs lors de la création de compte membre (deux auth_user_id pour la même personne).

Scénario : *Chef ajoute 'Nofane AMDJAD' → clic accidentel double → deux lignes team_members identiques → le membre apparaît deux fois dans l'UI, et deux tentatives de création de compte Supabase Auth seront faites pour le même email.*

```
supabase/migrations/equipe_schema.sql — CREATE TABLE team_members (pas de UNIQUE)
src/app/client/equipe/page.tsx — addMember() sans dédoublonnage
```

4. Findings P1 — Importants

Les findings P1 représentent des fonctionnalités essentielles manquantes ou des problèmes de sécurité secondaires à corriger rapidement après les P0.

P1 #6 — Mot de passe affiché en clair dans l'UI après création de compte

Lorsqu'un chef crée un compte membre, l'API retourne {email, password} et le composant stocke ces données dans l'état React **credResult**. Le mot de passe est ensuite affiché en clair dans un modal pour 'transmission au membre'. C'est une pratique à haut risque : le mot de passe peut être screenshoté, copié dans un log, ou exposé via les DevTools. À la place, l'invitation devrait se faire par email avec un lien de définition de mot de passe (Supabase Auth invite flow).

Scénario : *Chef crée membre → mot de passe généré '!Secure123' → affiché dans modal → chef fait Ctrl+A, Ctrl+C sur la page → mot de passe dans le presse-papier → potentiellement transmis en clair par email ou chat.*

`src/app/client/equipe/page.tsx — credResult state, modal d'affichage du mot de passe`

P1 #7 — chefId vérifié uniquement par comparaison avec le token (IDOR potentiel)

La route POST /api/equipe/create-member-account vérifie `caller.id === chefId` où chefId est envoyé dans le body de la requête par le frontend. Bien que la vérification soit correcte dans ce cas précis (on compare avec l'auth token serveur), le pattern reste fragile : si le frontend envoyait chefId=null ou un autre ID, le comportement dépendrait de la comparaison. La bonne pratique est de ne jamais accepter un identifiant d'identité du frontend — utiliser uniquement caller.id (issu du token).

Scénario : *Frontend envoie body: {chefId: 'autre-user-id', memberName: 'Hacker'} → la comparaison caller.id === 'autre-user-id' échoue → 403, mais le pattern reste fragile si la logique évolue.*

`src/app/api/equipe/create-member-account/route.ts — const { chefId } = await req.json()`

P1 #8 — Aucun pont entre team_members et organization_members

Créer un compte pour un membre via /api/equipe/create-member-account crée bien un utilisateur Supabase Auth et lie auth_user_id à team_members, mais ne crée **aucune entrée** dans **organization_members**. Or l'espace membre (/membre/*) repose entièrement sur organization_members pour l'auth, les permissions, et la navigation. Un membre dont le compte est créé depuis /client/equipe/ ne peut pas se connecter à /membre/login car il n'apparaît dans aucune organisation.

Scénario : *Chef crée compte pour 'Malika FARDI' → auth_user_id lié dans team_members → Malika tente /membre/login → POST /api/membre/login vérifie organization_members → 0 résultats → message 'Aucun espace membre trouvé' → Malika ne peut pas se connecter.*

`src/app/api/equipe/create-member-account/route.ts — aucun INSERT dans organization_members`
`src/app/membre/login/page.tsx — query sur organization_members uniquement`

P1

#9 — employes (paie) entièrement silotée de team_members

La table **employes** du module Paie est une table indépendante avec ses propres colonnes (nom, poste, salaire_brut, date_embauche, type_contrat). Elle n'a aucune FK vers team_members ou organization_members. Résultat : un chef doit saisir les mêmes personnes deux fois — une fois dans Équipe et une fois dans Paie. Les données peuvent diverger. Il n'y a aucune source de vérité unique pour les employés.

Scénario : *Chef ajoute 'Ahmed BENALI' dans team_members → veut gérer sa paie → aucun 'Ahmed BENALI' n'apparaît dans Paie → doit re-saisir manuellement → risque d'erreur de nom ou de prénom → données incohérentes entre Équipe et Paie.*

`supabase/migrations/employes_schema.sql — table employes sans FK vers team_members`

`src/app/client/equipe/page.tsx — aucune connexion avec la table employes`

5. Findings P2 — Améliorations

P2 **#10 — IA Équipe appelle /api/notes/ai (non dédié)**

La fonction runAI() dans /client/equipe/ appelle /api/notes/ai plutôt qu'un endpoint dédié à l'équipe. Le contexte injecté est générique (notes) et ne connaît pas les membres, les tâches d'équipe ou les congés.

`src/app/client/equipe/page.tsx — runAI() → /api/notes/ai`

P2 **#11 — Realtime Supabase non câblé dans /client/equipe/chat**

Le chat dans /client/equipe/ utilise une subscription Supabase Realtime pour les nouveaux messages (postgres_changes INSERT sur team_messages). Mais /membre/chat utilise aussi Realtime sur team_messages. Aucun des deux n'utilise les canaux org_messages — la vraie messagerie org ne bénéfice pas de temps réel côté chef.

`src/app/membre/chat/page.tsx — supabase.channel() sur team_messages (cassé)`

P2 **#12 — Organisation (organigramme) : aucune hiérarchie en base de données**

L'onglet Organisation de /client/equipe/ affiche une liste plate de membres sans aucune structure de départements ou de services en base. L'UI représente un organigramme mais les données n'ont aucune FK parent/enfant, aucune table departments/services.

`src/app/client/equipe/page.tsx — renderOrga() — affichage visuel seulement`

P2 **#13 — Évaluations employe_evaluations non scopées par organisation**

La table employe_evaluations est requêtée avec .eq('user_id', uid) uniquement. Aucun organization_id. Si deux chefs ont le même employé (hypothétique), les évaluations pourraient se croiser. Le module n'est pas préparé au multi-tenant.

`src/app/client/equipe/page.tsx — employe_evaluations query`

P2 **#14 — Flux d'invitation par email absent (mot de passe en clair uniquement)**

Supabase Auth expose supabase.auth.admin.inviteUserByEmail() qui envoie un lien d'invitation sécurisé. Cette API n'est pas utilisée. À la place, un mot de passe généré est affiché en clair (voir P1 #6). L'invitation par email offrirait plus de sécurité et une meilleure expérience membre.

`src/app/api/equipe/create-member-account/route.ts — signUp avec password`

6. Findings P3 — Finitions UX

P3 **#15 — Couleur SKY (#0ea5e9) au lieu de DJAMA GOLD dans /client/equipe/**

Toute l'interface de /client/equipe/ utilise const SKY = '#0ea5e9' comme couleur d'accent principale (avatars, boutons, badges, KPIs). Les autres modules ERP utilisent GOLD (#c9a55a). L'espace membre /membre/* utilise GOLD. Incohérence visuelle majeure dans la charte DJAMA.

`src/app/client/equipe/page.tsx — const SKY = '#0ea5e9' utilisé partout`

P3 **#16 — KPI bar affiche '0 EN COURS / 0 EN RETARD / 0 EN CONGÉ' en permanence**

Les KPIs en haut de l'écran comptent les tâches de team_tasks et les congés de team_leaves. Ces counts sont correctement calculés depuis les données chargées, mais comme il s'agit du modèle solo (user_id = chef), si le chef n'a pas de tâches assignées à lui-même, les compteurs restent à 0. Confusant pour l'utilisateur.

`src/app/client/equipe/page.tsx — KPI bar, calcul depuis tasks/leaves filtrés`

P3 **#17 — Canaux chat hardcodés (constante CLIENT-SIDE)**

const CHANNELS = ['général', 'annonces', 'projets', 'ressources'] est défini en dur côté client. Il est impossible d'ajouter, renommer ou supprimer un canal sans modifier le code. Pas de persistance, pas de customisation par organisation.

`src/app/client/equipe/page.tsx — const CHANNELS; src/app/membre/chat/page.tsx — const CHANNELS`

P3 **#18 — Cartes membre : informations redondantes, affichage non condensé**

Les cartes membres dans la grille répètent l'avatar, le nom et le rôle de façon identique pour chaque membre sans différenciation visuelle. Le statut (en ligne, last_seen_at) n'est pas affiché. La gestion de duplicats visuels (Nofane x2) n'est pas traitée UI.

`src/app/client/equipe/page.tsx — renderMembers() grid`

7. Audit Supabase / RLS — Table par Table

Évaluation des politiques RLS pour chaque table du module Équipe.

Table	org_id	user_id	RLS	Politique principale	Risque
team_members	■ Non	■ Oui	■ Oui	user_id = auth.uid() — chef seulement	CRITIQUE
team_tasks	■ Non	■ Oui	■ Oui	user_id = auth.uid() — brise /membre/taches	CRITIQUE
team_messages	■ Non	■ Oui	■ Oui	user_id = auth.uid() — brise /membre/chat	CRITIQUE
team_leaves	■ Non	■ Oui	■ Oui	user_id = auth.uid() — chef seulement	ÉLEVÉ
team_meetings	■ Non	■ Oui	■ Oui	user_id = auth.uid() — chef seulement	ÉLEVÉ
organization_members	■ Oui	■ Oui	■ Oui	is_org_member() SECURITY DEFINER	BAS
organization_permissions	■ Oui	■ Oui	■ Oui	is_org_admin() pour write, is_org_member() pour read	BAS
org_messages	■ Oui	■ Oui	■ Oui	is_org_member() + group membership	BAS
org_message_groups	■ Oui	■ Oui	■ Oui	is_org_member() SECURITY DEFINER	BAS
org_shared_documents	■ Oui	■ Oui	■ Oui	can_view_shared_doc() helper function	BAS
employes	■ Non	■ Oui	■ Oui	user_id = auth.uid() — paie isolée	MOYEN
employe_evaluations	■ Non	■ Oui	■ Oui	user_id = auth.uid() — chef seulement	MOYEN
timesheets	■ Non	■ Oui	■ Oui	user_id = auth.uid() — chef seulement	ÉLEVÉ

Note : La colonne org_id indique si la table possède une colonne organization_id. Les tables team_* n'en ont aucune, ce qui est la cause racine du P0 #1. Les tables org_* ont toutes un organization_id et des RLS robustes.

8. Architecture Membre et Permissions

8.1 Flux d'authentification actuel

1. Création membre (chef)	`/api/equipe/create-member-account` → `supabase.auth.admin.createUser()` → `INSERT team_members.auth_user_id` → ■■ PAS d'INSERT dans `organization_members`
2. Login membre	`/membre/login` → `supabase.auth.signInWithPassword()` → query `organization_members` → `suspended_at IS NULL` → sélection `active_org_id` → stockage `user_metadata`
3. Navigation membre	`/membre/layout.tsx` → query `organization_members` + `organization_permissions` → construit nav dynamique → vérifie suspension
4. Pages membres	Lisent org_* (dashboard OK, messages OK) ou team_* (taches ■, chat ■) selon la page

8.2 Schema organization_permissions

La table organization_permissions possède les colonnes : `(organization_id, user_id, app_slug, can_view, can_create, can_edit, can_delete, can_export)`. Elle est correctement scopée par org et protégée par RLS. Cependant, son usage est limité à l'UI (sidebar) et n'est pas appliqué côté API — voir P0 #4.

app_slug	can_view	can_create	can_edit	can_delete	can_export	API protégée ?
factures	UI	UI	UI	UI	UI	■ Non
crm	UI	UI	UI	UI	UI	■ Non
documents	UI	UI	UI	UI	UI	■ Non
planning	UI	UI	UI	UI	UI	■ Non

9. Connexions ERP

Analyse des connexions entre le module Équipe et les autres modules DJAMA ERP.

Module	Direction	Connexion actuelle	État
Planning	→ Équipe	planning_events a team_member_id (nullable). Overlay tâches dans agenda.	■ Partiel
Productivité	→ Équipe	productivity_tasks a assigned_to (team_member_id). Tâches équipe assignées.	■ Partiel
Paie / RH	Équipe ↔ Paie	Aucune FK. employes et team_members sont des tables indépendantes.	■ Cassé
CRM	— aucune —	Pas de connexion directe entre contacts CRM et membres équipe.	■ Absent
Factures/Devis	— aucune —	Pas d'assignation de factures à des membres équipe.	■ Absent
Projets	→ Équipe	projects a team_members_ids[] (array). Membres assignés aux projets.	■ Partiel
Documents	→ Équipe	org_shared_documents pour partage avec membres. Bien structuré.	■ OK
Messagerie org	Équipe ↔ Msg	org_messages/org_message_groups pour les membres. Chef utilise team_messages.	■ Cassé
Coaching IA	— aucune —	Le coaching IA est individuel, pas lié aux membres équipe.	■ Absent

- **Connexions fonctionnelles :** org_shared_documents (documents partagés) → état satisfaisant
- **Connexions partielles :** Planning, Productivité, Projets ont des FK vers team_members mais sans synchronisation avec organization_members
- **Connexion cassée critique :** Paie/RH (employes ≠ team_members) — registres entièrement séparés
- **Connexion chat cassée :** team_messages (chef) ≠ org_messages (membres) — silots non synchronisés

10. Audit Design / UX

Composant	État	Observations
Couleur principale	■ Problème	SKY (#0ea5e9) utilisé partout dans /client/equipe/. Tous les autres modules DJAMA utilisent GOLD (#c9a55a). Incohérence de charte.
Navigation onglets	■ Bon	7 onglets (Membres, Tâches, Chat, RH, Orga, Évals, IA) avec icônes cohérentes. Navigation claire.
Cartes membres	■ Améliorable	Grille responsive. Mais doublons visibles (Nofane x2 confirmé). last_seen_at non affiché. Statut en ligne absent.
KPI Bar	■ Problème	Affiche '0 EN COURS / 0 EN RETARD / 0 EN CONGÉ' en permanence (données non scopées correctement).
Kanban tâches	■ Bon	3 colonnes (À faire, En cours, Terminé) avec glisser-déposer visuel, ajout rapide, assignation membre, date d'échéance.
Chat canaux	■ Problème	Canaux hardcodés. Design propre mais données cassées (voir P0 #3). Pas de groupements de dates.
Tab RH	■ Améliorable	Calendrier d'absences bien conçu avec overlay congés. Timesheets avec export CSV. Réunions planifiées. Fonctionnel mais données non org-scopées.
Tab Organigramme	■ Améliorable	Affichage arborescent des membres sans hiérarchie DB. Visuellement correct mais statique.
Tab Évaluations	■ Améliorable	Formulaire d'évaluation avec notation 1-5. Pas de workflow d'approbation. Pas de confidentialité manager/employé.
Tab IA	■ Bon	Interface IA avec suggestions rapides. Contexte enrichi. Utilise GOLD pour les accents (bon). Mais appelle /api/notes/ai (non dédié équipe).
Espace membre /membre/*	■ Bon	Design GOLD cohérent, navigation responsive, dark mode, sidebar drawer mobile. Bien conçu.
Mobile responsiveness	■ Partiel	/client/equipe/ : pas de sidebar mobile drawer (pas de hamburger). /membre/* : responsive complet avec drawer.
États vides	■ Manquant	Les onglets RH, Orga, Évals n'ont pas d'états vides informatifs quand les listes sont vides.
Dark mode	■ Bon	isDark toggle cohérent. Palettes dark/light correctement alternées dans les deux frontends.

11. Roadmap Complète

Roadmap priorisée pour la refonte du module Équipe. Organisée en phases progressives pour livrer de la valeur rapidement tout en résolvant les blocages fondamentaux.

Phase 1 — P0 Urgents (semaine 1-2)

Phase	Action	Sévérité	Effort	Impact
1.1	Contrainte UNIQUE (user_id, email) sur team_members	P0	S	Élimine les doublons de membres
1.2	Migration : ajouter organization_id sur team_* tables	P0	M	Prépare l'isolation multi-tenant
1.3	Créer un pont team_members ↔ organization_members	P0	L	Membres créés → accès /membre/* fonctionnel
1.4	Corriger RLS team_tasks pour les membres	P0	S	Page /membre/taches affiche les vraies tâches
1.5	Unifier chat : tout passer par org_messages	P0	L	Chef et membres dans le même canal

Phase 2 — P1 Sécurité et fonctionnalité (semaine 3-4)

Phase	Action	Sévérité	Effort	Impact
2.1	Flux invitation par email (supabase.auth.admin.inviteUserByEmail)	P1	M	Fin du mot de passe en clair
2.2	Vérification organization_permissions côté API (middleware)	P1	L	Permissions appliquées server-side
2.3	Lier employes ↔ team_members (FK ou view unifiée)	P1	M	Source de vérité unique pour les employés
2.4	Supprimer chefId du body, utiliser uniquement caller.id	P1	S	Sécurité API create-member-account

Phase 3 — P2 Améliorations (semaine 5-6)

Phase	Action	Sévérité	Effort	Impact
3.1	Endpoint dédié /api/equipe/ai avec contexte équipe	P2	M	IA contextuelle : membres, tâches, congés
3.2	Supabase Realtime sur org_messages (chef + membres)	P2	M	Chat en temps réel bidirectionnel
3.3	Table departments/services avec FK parent/enfant	P2	M	Organigramme hiérarchique persisté en DB

Phase	Action	Sévérité	Effort	Impact
3.4	organization_id sur employe_evaluations	P2	S	Évaluations scoped par organisation
3.5	Mobile hamburger sidebar pour /client/equipe/	P2	S	Responsivité complète chef

Phase 4 — P3 Finitions UX (semaine 7-8)

Phase	Action	Sévérité	Effort	Impact
4.1	Remplacer SKY par GOLD dans /client/equipe/	P3	XS	Cohérence charte DJAMA
4.2	Canaux chat configurables en DB (channels table)	P3	M	Personnalisation par organisation
4.3	KPI bar corrigée avec vraies données org-scoped	P3	S	Compteurs significatifs
4.4	États vides informatifs sur tous les onglets	P3	S	UX professionnelle

Effort estimé : XS = < 1h | S = 1-4h | M = 0.5-1j | L = 1-3j | XL = 3j+

12. Risques et Recommandations

RISQUE CRITIQUE

Migration des tables team_*

Risque : L'ajout d'organization_id sur team_* est une migration destructive si elle n'est pas idempotente. Des données existantes peuvent être perdues si la migration suppose que tous les enregistrements appartiennent à une org spécifique.

Mitigation : *Rendre la colonne NULLABLE au départ. Backfill via un script séparé. Tester sur données réelles avant.*

RISQUE ÉLEVÉ

Unification chat team_messages → org_messages

Risque : Si des messages historiques existent dans team_messages, ils seront perdus lors de la migration vers org_messages. Les membres verront un historique vide.

Mitigation : *Script de migration qui copie team_messages vers org_messages avec sender_name et org_id. Conserver team_messages en lecture seule pendant une période de transition.*

RISQUE ÉLEVÉ

Flux d'invitation par email

Risque : La migration de 'création avec mot de passe' vers 'invitation par email' va casser les comptes membres déjà créés (ils ont un mot de passe, pas un lien d'invitation).

Mitigation : *Garder les deux flows en parallèle. Nouveaux comptes → invitation email. Existants → reset password forcé.*

RISQUE MOYEN

Middleware organization_permissions sur les API

Risque : Ajouter une vérification des permissions sur toutes les routes API peut introduire des régressions si des membres utilisent des routes qu'ils n'ont pas explicitement. Des permissions par défaut trop restrictives peuvent casser des workflows existants.

Mitigation : *Définir des permissions par défaut permissives (can_view: true pour tous). Logger les refus pendant 2 semaines avant d'activer le blocage.*

13. Conclusion

Le module Équipe de DJAMA ERP contient des fondations solides dans sa nouvelle architecture multi-tenant (espace membre /membre/*, org_* tables, RLS robuste, suspension, permissions dynamiques), mais souffre d'une **dette technique critique** liée à la coexistence non résolue de deux paradigmes : l'ancien modèle solo (`team_*`) et le nouveau modèle multi-tenant (`org_*`).

Cette fracture architecturale rend 3 pages de l'espace membre non fonctionnelles, crée une faille de sécurité sur les permissions API, et génère une confusion de données (membres dupliqués, registres employés séparés). Ces problèmes existent indépendamment et cumulativement — ils doivent être traités comme un ensemble cohérent plutôt que comme des bugs isolés.

Dimension	Évaluation	Détail
Sécurité Auth	■ Moyen	Authentification Supabase correcte mais permissions API bypassables
RLS Isolation	■ **Critique**	team_* sans organization_id — isolation multi-tenant absente
Espace Membre	■ **Cassé**	3 pages cassées (/taches, /chat, /messages partiel)
Architecture	■ Fragile	Deux systèmes parallèles sans pont — refonte nécessaire
Design / UX	■ Partiel	Bonne base mais SKY vs GOLD, KPIs à 0, états vides manquants
Connexions ERP	■ Partiel	Documents OK, Paie silotée, Chat cassé, autres partiels
Score global	**3.5 / 10**	Potentiel fort, fondation multi-tenant correcte, mais blocages critiques à résoudre

Prochaine étape : soumettre pour autorisation d'implémentation. Les phases 1 et 2 de la roadmap (P0+P1) sont estimées à 2-3 semaines de développement. La phase 1 seule (P0 urgents) peut être livrée en 1 semaine avec des migrations idempotentes et sans destruction de données existantes.

Audit réalisé le 29 septembre 2026 DJAMA Premium — Confidentiel nofane.soufie@gmail.com